

# Mock Design: A Distributed Cache

Phase 4 · Interview-Ready · Estimated time: 1.5 hours

---

## 1. Concept From First Principles: Applying the Framework

"Design a distributed cache" is a different kind of prompt than the previous two chapters: instead of designing a product feature, you're asked to design the infrastructure component itself — the thing Phase 1 Chapter 8 treated as a given building block. This inversion is exactly why it's a common senior-level question: it tests whether you understand what's inside the box you've been relying on, not just how to use it.

### Step 1 — Clarify requirements

- Functional: `get(key)`, `set(key, value, ttl)`, `delete(key)`; support eviction when memory is full.
- Non-functional: extremely low latency (sub-millisecond reads are the entire point of a cache existing); horizontal scalability across many nodes as data volume grows beyond one machine's RAM; graceful behavior under node failure.

### Step 2 — Estimate scale

Assume 10 million distinct hot keys, average value size 1KB, giving ~10GB of hot data — more than comfortably fits on one machine's RAM at scale when you also want replication headroom, so multiple nodes are required. Assume 1 million reads/second at peak, which single-node Redis could theoretically handle in isolation, but the interview is specifically asking you to design for horizontal scale and fault tolerance beyond a single node.

### Step 3 — API

A simple client library interface: `get(key) -> value | miss`, `set(key, value, ttl)`, `delete(key)`. The interesting design work is entirely inside how these three calls route to the correct node and stay consistent — not the interface shape itself.

### Step 4 — High-level design

Multiple cache nodes, each holding a partition of the total key space in RAM. A client (or a thin routing layer) must know which node owns a given key. The naive approach — `hash(key) % num_nodes` — breaks badly when nodes are added or removed: nearly every key's target node changes, causing a massive simultaneous cache-miss stampede against the backing database. The correct answer is **consistent hashing** (Phase 1, Ch 8): nodes and keys are placed on a hash ring, and adding or removing one node only remaps the keys immediately adjacent to it on the ring, minimizing redistribution. Each node also replicates its data to 1–2 other nodes for fault tolerance, so a single node failure doesn't lose that data entirely.

### Step 5 — Deep dive: eviction policy and consistency with the source of truth

**Eviction:** when a node's memory fills up, something must be evicted to make room. **LRU (Least Recently Used)** is the most common default — evict the key that hasn't been accessed longest,

implemented efficiently with a hash map plus doubly-linked list giving  $O(1)$  access and eviction. **LFU (Least Frequently Used)** is sometimes better for workloads where popularity, not recency, predicts future access, but is more expensive to track accurately. Name the trade-off rather than just the name: LRU can evict a genuinely popular key that simply wasn't accessed in the last few minutes, while LFU can get stuck favoring historically popular keys that are no longer actually hot.

**Cache-database consistency** is the harder, more senior-signaling deep dive. **Cache-aside (lazy loading)**: application checks cache first, on miss reads the database and populates the cache — simple, and a cache outage degrades to direct database reads rather than failing outright.

**Write-through**: every write goes to the cache and database together, synchronously — keeps the cache always fresh but adds write latency. **Write-behind (write-back)**: writes go to the cache immediately and are asynchronously flushed to the database later — fastest writes, but risks data loss if the cache node crashes before the flush completes. Most production systems favor cache-aside for reads combined with a short TTL or explicit invalidation on write, specifically because it degrades gracefully rather than introducing a new durability risk.

## Step 6 — Trade-offs and wrap-up

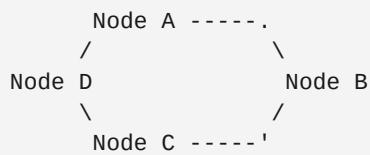
Volunteer: the classic **thundering herd / cache stampede** problem — when a very hot key expires or a new node comes online, many concurrent requests can simultaneously miss and all hit the database at once. Mitigate with request coalescing (only one request per key actually queries the database while others wait) or staggered TTLs to avoid mass-simultaneous expiry.

## 2. Real-World Example / Case Study

Redis and Memcached, used extensively across Flipkart's, Swiggy's, and most Tier-1 product companies' backend stacks, are real production instances of exactly this design, and understanding consistent hashing and cache-aside vs. write-through is directly applicable to how these teams actually operate their caching layers at scale, not just an abstract interview exercise.

## 3. Diagram

Consistent hash ring, 4 cache nodes:



key "user:4521" hashes to a ring position ->  
walks clockwise -> lands on Node B (owner)  
(replicated to Node C for fault tolerance)

Cache-aside read path:

```
[App] --get(key)--> [Cache] --miss--> [Database]
                        ^               |
                        '----- populate ----'
```

Write-through write path:

```
[App] --set(key,val)--> [Cache] --sync write--> [Database]
                        (both succeed before ack)
```

## 4. Trade-offs Table

| Decision                           | Alternative                        | Why this choice                                                                    |
|------------------------------------|------------------------------------|------------------------------------------------------------------------------------|
| Consistent hashing for key routing | <code>hash(key) % num_nodes</code> | Minimizes key redistribution when nodes are added/removed                          |
| LRU eviction                       | LFU eviction                       | Cheaper to track accurately; good default unless popularity truly outlasts recency |
| Cache-aside for reads              | Write-through                      | Degrades gracefully to direct DB reads on cache outage, no new write-path risk     |
| Request coalescing for hot keys    | No stampede protection             | Prevents a single expiring key from causing a simultaneous DB traffic spike        |

## 5. Common Interview Questions

### Q1: "What happens to your cache when you add a new node to the cluster?"

Model approach: Explain consistent hashing directly — only the keys immediately adjacent to the new node's ring position get remapped, versus a naive modulo scheme where nearly every key remaps simultaneously, causing a mass cache-miss stampede.

### Q2: "Cache-aside vs write-through — when would you choose each?"

Model approach: Cache-aside for read-heavy workloads where graceful degradation on cache failure matters most; write-through when the cost of ever serving stale data is high enough to justify the added write latency, since it keeps the cache always in sync with every write.

### Q3: "How do you prevent a cache stampede when a very popular key expires?"

Model approach: Name request coalescing (one in-flight database request per key, others wait on it) and/or staggered/jittered TTLs so many keys don't expire in the exact same instant, both directly

preventing the simultaneous-miss scenario.

## 6. Practice Exercises

- 1 Run the full six-step framework for "design a distributed cache," out loud, with a timer.
- 2 Implement LRU eviction's core data structure (hash map + doubly linked list) in pseudocode, and explain why it achieves  $O(1)$  get and evict.
- 3 Explain, from memory, the cache stampede problem and two mitigations, without notes.

## 7. Curated YouTube Videos

| Language | Video & Channel                                                                                     | Why it helps                                                                                |
|----------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| English  | "System Design Interview - Design a Distributed LRU Cache (Full mock interview with Sr. MAANG SWE)" | Full live mock interview matching this chapter's exact structure.                           |
| English  | "System Design Interview - Distributed Cache"                                                       | Covers consistent hashing and eviction policy trade-offs directly.                          |
| Hindi    | "Redis system design   Distributed cache System design"                                             | Hindi/Hinglish coverage of distributed cache internals using Redis as the concrete example. |
| Hindi    | "System Design Complete Course 2026" (Hindi + English, 10+ hrs)                                     | Bilingual course covering caching as part of the broader curriculum.                        |

## 8. Key Takeaways

- This chapter designs the caching infrastructure itself, reversing Phase 1 Ch 8's perspective of treating a cache as a given building block.
- Consistent hashing (not naive modulo) is required to avoid mass key redistribution when the cluster resizes.
- LRU is the common default eviction policy; LFU trades implementation cost for better handling of genuinely popularity-driven workloads.
- Cache-aside degrades gracefully on cache failure; write-through keeps data always fresh at the cost of write latency.
- Cache stampede (thundering herd) is a real, nameable failure mode with concrete mitigations: request coalescing and staggered TTLs.
- Replication across nodes protects against single-node data loss, on top of consistent hashing's partitioning.
- Naming the specific trade-off behind each choice, not just the pattern's name, is what separates a strong deep-dive answer here.